Ben-Gurion University of the Negev

Faculty of Engineering Sciences

Department of Software and Information systems

Deep Reinforcement Learning

Assignment #3

By

Oz Elbaz – 204388763

Ido Gurevich - 205478068

Section 1 - Training individual networks:

In this section, we trained three separate A2C agents, each on a different Gym environment: **CartPole-v0**, **Acrobot-v1**, and **MountainCarContinuous-v0**.

For each agent, we will present the following:

1. **Architecture** – The design of the neural network.
2. **Hyperparameter Tuning** – The process of optimizing model parameters.
3. **Results** – A performance comparison to determine which agent outperforms the others.

Additionally, to facilitate future transfer learning, we standardized the input and output dimensions across all agents. The state vector was set to a universal size of six, matching the largest state space among the three environments. Similarly, the actor's output was defined as a vector of size three, corresponding to the highest number of possible actions across these environments.

For tunning the hyper-parameters, we've used "**Optuna**", which is a python package for optimizing neural networks hyper-parameters, using iterative sampling process along a range of values of each hyper-parameter.

1. A2C 'Cartpole-v1':

In the '**CartPole-v1'** environment, the agent receives a reward of **+1** for every time step that the pole remains balanced and does not fall . Each episode was defined with maximum of 500 steps . To implement early stopping during training, we set a criterion where the average reward over the last **100 episodes** must exceed **475**. Once this condition is met, training is stopped early.

Architecture:

Actor:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**12**	**Relu**
2	**Linear**	**12**	**Relu**
3	**Linear**	**12**	**-**

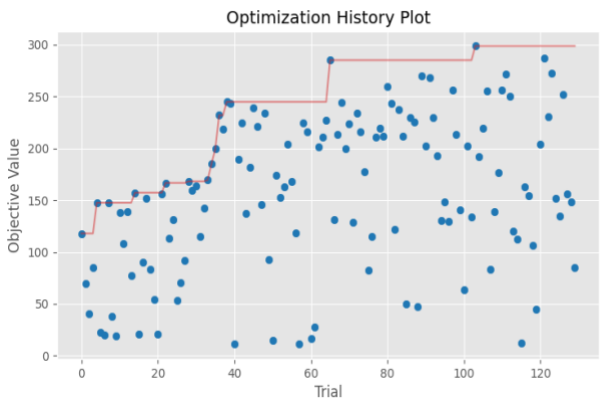
Table 1: Actor's architecture, 'cartpole-v1'

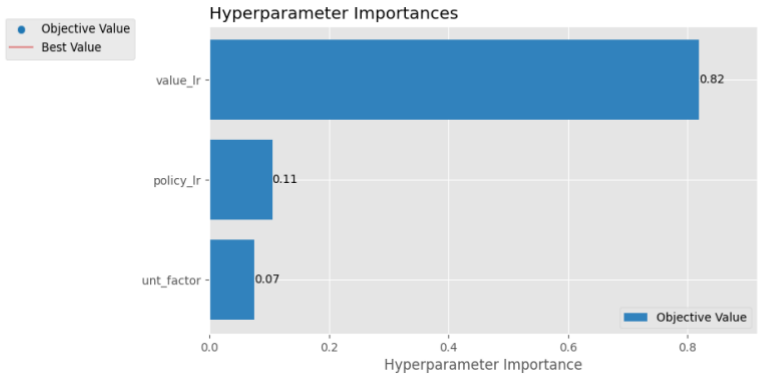
Critic:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**64**	**Relu**
2	**Linear**	**64**	**Relu**
3	**Linear**	**64**	**Softmax**

Table 2: Actor's architecture, 'Cartpole-v1'.

Hyperparameter tuning:





In the optimization history graph, each blue dot represents a "trial" which is a run of 1000 episodes. The Y axis represent the objective value, which we defined as average rewards on the entire trial. The graph shows the improvement in selecting the hyper-parameter, although the objective values are not all correlated with the best values as they should, which can state on high variations between hyper parameters values.

The Hyperparameter importance graph, shows which of the hidden parameters has the most impact on the model's performance. In this case, we can see it is the learning rate of the value network which affect the most. This can state on poor selection of the policy network architecture.

Results:

Ultimately, we selected these hyperparameters because this agent outperformed the others. The chosen hyperparameters are:

- **Discount factor:** 0.99
- **Actor/Policy learning rate:** 0.0001
- **Critic/Value network learning rate:** 0.005

As shown in the figure, the agent's reward per episode stabilized after approximately **700 episodes**, indicating that the agent converged around that point. The total time it took for the agent to converge was roughly **161 seconds**.

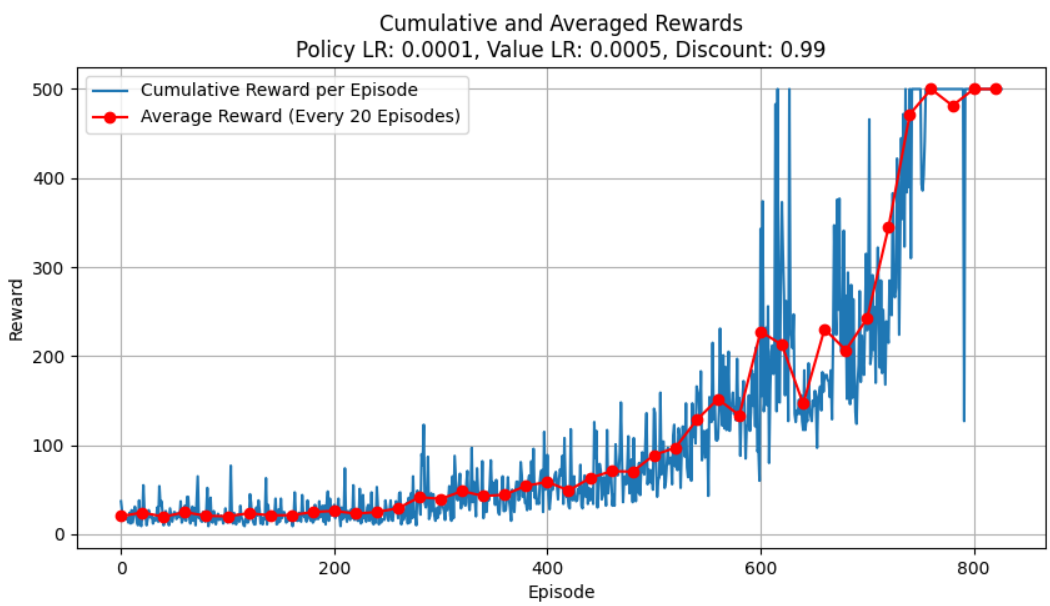


Figure 2: The best agent. The blue line represents the exact reward per episode and the red line represent the averaged reward of the previous 20 episodes.

2. *A2C 'Acrobot-v1':*

In the '**Acrobot-v1'** environment, the agent receives a reward of **-1** for every time step that the pole remains in the lower side of the environment. Each episode was defined with maximum of 500 steps. To implement early stopping during training, we set a criterion where the average reward over the last **100 episodes** must exceed **-100**. Once this condition is met, training is stopped early.

Architecture:

Actor:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**12**	**Relu**
2	**Linear**	**12**	**Relu**
3	**Linear**	**12**	**-**

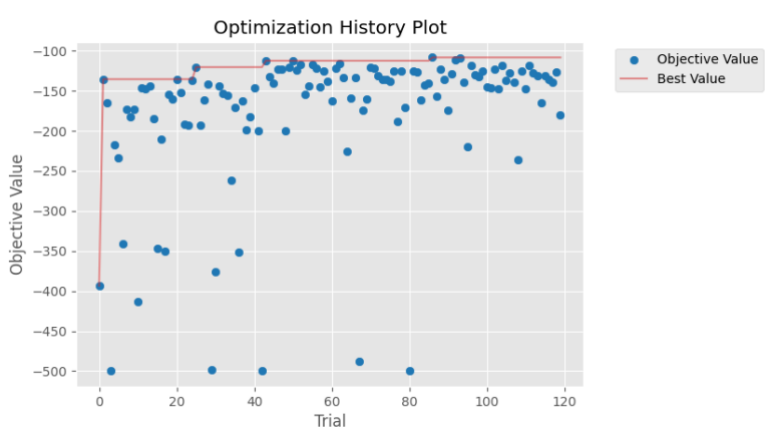
Table 3: Actor's architecture, 'Acrobot-v1'

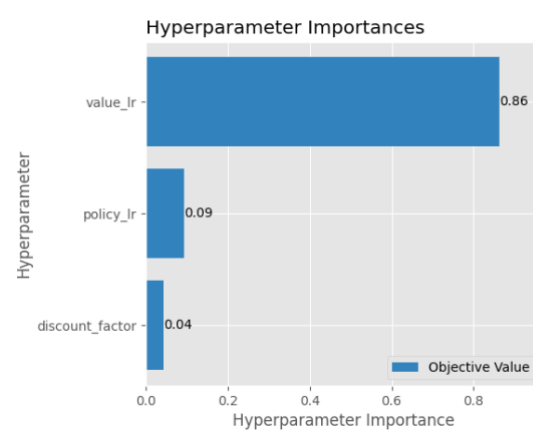
Critic:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**64**	**Relu**
2	**Linear**	**64**	**Relu**
3	**Linear**	**64**	**-**

Table 4: Actor's architecture, 'Acrobot-v1'.

Hyperparameter tuning:





Results:

Ultimately, we selected these hyperparameters because this agent outperformed the others. The chosen hyperparameters are:

- **Discount factor:** 0.99
- **Actor/Policy learning rate:** 0.001
- **Critic/Value network learning rate:** 0.001

As shown in the figure, the agent's reward per episode stabilized after approximately **20 episodes**, indicating that the agent converged around that point. The total time it took for the agent to converge was roughly **24 seconds**.

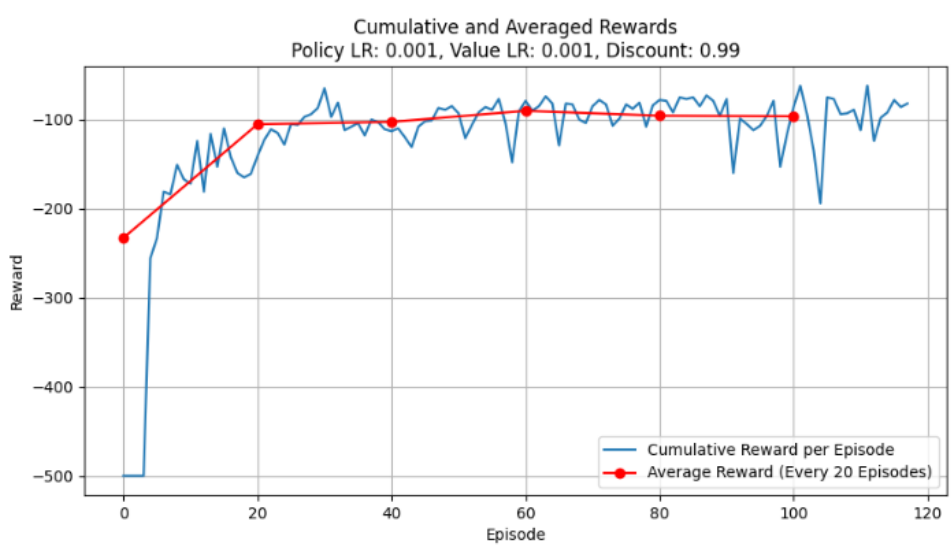


Figure 4: The best agent of the acrbot-enviroment. The blue line represents the exact reward per episode and the red line represent the averaged reward of the previous 20 episodes.

3. *A2C 'Mountaincarcontinous-v0':*

In the **MountainCarContinuous-v0** environment, the reward structure presents a challenge. If the agent fails to reach the right hill at least once early in training, it may quickly learn to remain stationary. This behavior leads the agent to settle at a local maximum reward of **0**, missing the opportunity to achieve a much higher reward. Ideally, the agent should be encouraged to take larger actions initially, even if they result in short-term penalties, to eventually gain a significantly higher reward.

Each episode is limited to a maximum of **999 steps**. To implement early stopping, we set a criterion where the **average reward over the last 100 episodes** must exceed **50**. Once this condition is met, training stops early.

Additionally, to prevent the agent from getting stuck at the local maximum of zero, we reset the agent's network weights if it fails to reach the right hill at least once within **three consecutive episodes**. This safety measure helps encourage exploration and prevents the agent from adopting suboptimal strategies.

Architecture:

Actor:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**16**	**Relu**
2	**Linear**	**16**	**Relu**
3	**Linear**	**16**	**-**

Table 5: Actor's architecture, 'MountainCarContinous-v0'.

Critic:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**64**	**Relu**
2	**Linear**	**64**	**Relu**
3	**Linear**	**64**	**-**

Table 6: Actor's architecture, 'MountainCarContinous-v0'.

Results:

Ultimately, we selected these hyperparameters because this agent outperformed the others. The chosen hyperparameters are:

- **Discount factor:** 0.99
- **Actor/Policy learning rate:** 0.00001
- **Critic/Value network learning rate:** 0.000055

As shown in the figure, the agent's reward per episode stabilized, smaller oscillations, after approximately **60 episodes**, indicating that the agent converged around that point. The total time it took for the agent to converge was roughly **172 seconds**.

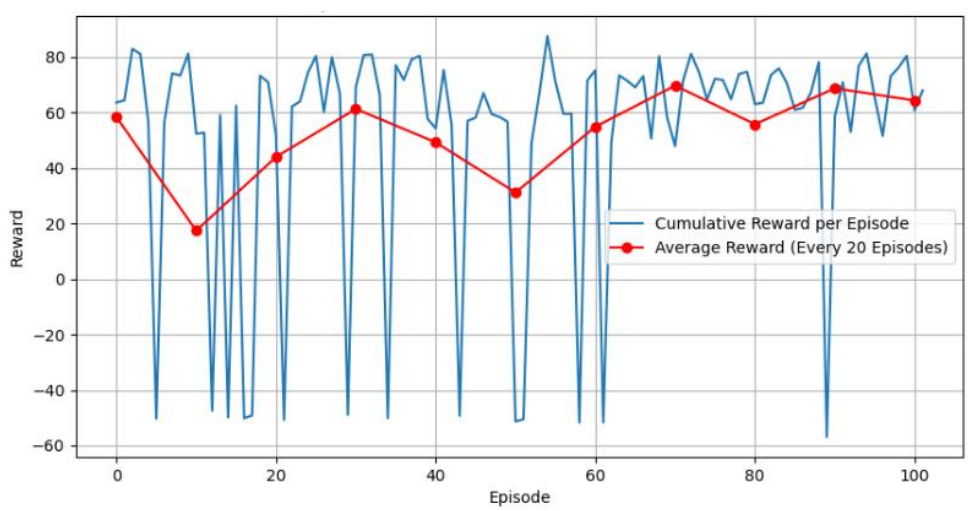


Figure 6: The best agent of the MountCarContinous-enviroment. The blue line represents the exact reward per episode and the red line represent the averaged reward of the previous 20 episodes.

Section 2 – Fine tune an existing model

In this section, we will **retrain the final layers** of two different models on new tasks that differ from their original training problems. First, we will retrain an agent originally trained on the **'Acrobot-v1'** environment to solve the **'CartPole-v1'** task. Next, we will apply the same process to an agent initially trained on the **'CartPole-v1'** environment, adapting it to the **'MountainCarContinuous-v0'** task. For each agent, we will cover the same key topics as in the previous section: **model architecture**, **hyperparameter tuning**, and **experimental results**.

1. 'Acrobot-v1' to 'Cartpole-v1':

Architecture:

Actor:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**12**	**Relu**
2	**Linear**	**12**	**Relu**
3	**Linear**	**12**	**Softmax**

Table 7: Actor's architecture, 'cartpole-v1'

Critic:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**64**	**Relu**
2	**Linear**	**64**	**Relu**
3	**Linear**	**64**	**-**

Table 8: Actor's architecture, 'Cartpole-v1'.

Hyperparameter tuning:

To optimize the agent's performance and achieve faster convergence, we experimented with several hyperparameter configurations, as detailed in **Table 9**. Initially, we applied the hyperparameters that were successful in training the agent on the Acrobot-v1 environment. This setup enabled the agent to achieve a reward of 500 per episode after approximately 700 episodes. However, despite this progress, the agent failed to fully converge within 1,000 episodes, with an average runtime of 287 seconds (Figure 7A).

Hyper parameters	Original Acrobot	Original Cartpole-v1	Attempt 1	Attempt 2	Attempt 3
Mark	**A**	**B**	**C**	**D**	*
Discount factor	**0.99**	**0.99**	**0.999**	**0.99**	**0.99**
Actor LR	**0.001**	**0.0001**	**0.001**	**0.0005**	**0.0008**
Critic LR	**0.001**	**0.0005**	**0.001**	**0.001**	**0.001**
Averaged Running time [sec]	**287**	**51**	**146**	**256**	**312**
Convergence (episodes)	**Didn't**	**Didn't**	**Didn't**	**Didn't**	**Didn't**

Table 9: Hyper-parameterization results, 'Acrobot-v1' to 'Cartpole-v1'.

Next, we tested the original hyperparameters designed for the CartPole-v1 environment. This attempt resulted in the weakest performance among all configurations. The agent struggled to balance the pole and failed to make significant learning progress throughout the training. The average runtime was only 51 seconds for 1,000 episodes, reflecting the agent's inability to maintain stability for a meaningful duration (Figure 7B).

To improve performance, we explored three additional sets of hyperparameters. Unfortunately, none of these configurations led to successful convergence within the given training limit. Despite this, the first attempt using the Acrobot-v1 hyperparameters consistently outperformed the other trials, demonstrating relatively better learning progression. A more in-depth analysis of these results will be presented in the **Results** section.

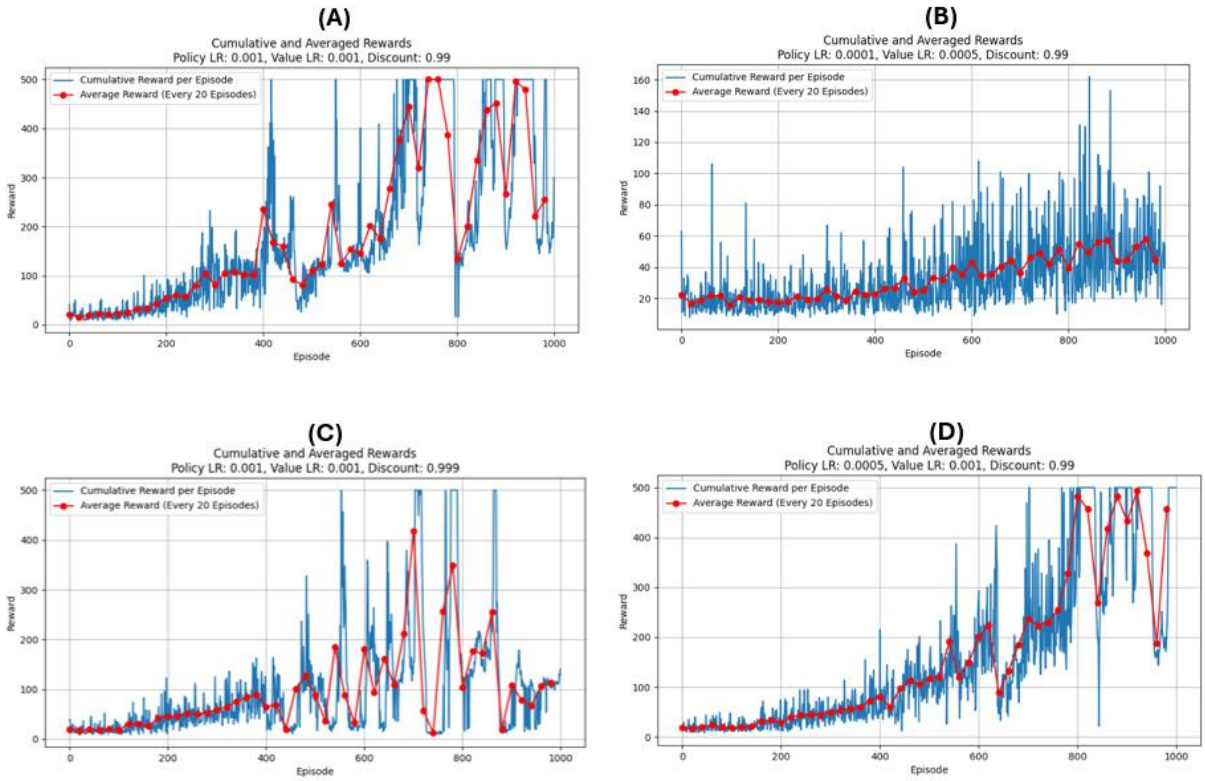
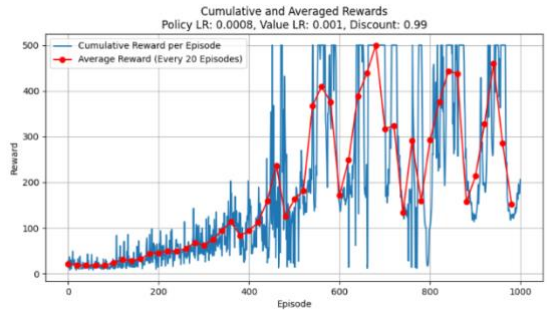


Figure 10: Performance of different agents on the CartPole-v1 environment. Each graph (A)–(D) corresponds to the hyperparameter settings detailed in Table 9. The cumulative rewards (blue) and averaged rewards over every 20 episodes (red) illustrate the learning progress of each agent.

Results:

Ultimately, we selected these hyperparameters because this agent outperformed the others, the third attempt. The chosen hyperparameters are:



- **Discount factor:** 0.99
- **Actor/Policy learning rate:** 0.0008
- **Critic/Value network learning rate:** 0.001

The agent managed to achieve a reward of 500 for the first time after approximately 500 episodes. Additionally, it had the longest average running time of 312 seconds, indicating that it took the most steps across 1,000 episodes. Despite showing better performance compared to the other attempts, this agent still failed to fully converge.

In summary, the model trained on the *Acrobot-v1* environment was unable to successfully transfer and converge in the *CartPole-v1* environment. This is likely due to the fundamental differences between the two tasks. The *Acrobot-v1* task involves controlling a double pendulum system with complex, nonlinear dynamics and requires swinging up to reach a target height. In contrast, *CartPole-v1* involves balancing a pole in a more linear and reactive control scenario. The skills and control strategies learned in *Acrobot-v1* do not directly apply to the balance-oriented dynamics of *CartPole-v1*, making transfer learning between these two tasks challenging. Differences in state space representation, control frequency, and the reward structures may have also contributed to the model's inability to adapt effectively.

2. ‘Cartpole-v1’ to ‘MountainCarContinous-v0’:

Architecture:

Actor:

Layer	Type of layer	Layer’s size	Activation function
1	**Linear**	**12**	**Relu**
2	**Linear**	**12**	**Relu**
3	**Linear**	**12**	**-**

Table 10: Actor’s architecture, ‘Cartpole-v1’ to ‘MountainCarContinous-v0’.

Critic:

Layer	Type of layer	Layer’s size	Activation function
1	**Linear**	**64**	**Relu**
2	**Linear**	**64**	**Relu**
3	**Linear**	**64**	**-**

Table 11: Critic’s architecture, ‘Cartpole-v1’ to ‘MountainCarContinous-v0’.

Hyperparameter tuning + results:

To optimize the agent's performance and achieve faster convergence, we tested several hyperparameter configurations, as summarized in Table 12. Initially, we applied the hyperparameters that had proven effective for training the agent in the 'CartPole-v1' environment. Subsequently, we experimented with the hyperparameters that yielded the best results for the individual network in the 'MountainCarContinuous-v0' environment. Following these attempts, we explored additional configurations, with the two best-performing sets highlighted in the table. Each agent was run 10 times to ensure reliability, and the average runtime and convergence performance were evaluated. Convergence was defined as achieving an average score of over 50 per episode across 100 consecutive episodes.

Hyper parameters	Original Cartpole-v1	Original Mountain-continous	Attempt 1	Attempt 2
Mark	**A**	**B**	**C**	**D**
Discount factor	**0.99**	**0.99**	**0.999**	**0.99**
Actor LR	**0.0001**	**0.00001**	**0.00001**	**0.00005**
Critic LR	**0.0005**	**0.000055**	**0.000055**	**0.00055**
Averaged Running time [sec]	**138**	**273**	**178**	**169**
Convergence (episodes)	**101**	**Didn't**	**105**	**103**

Table 12: Hyper-parameterization results , 'Cartpole-v1' to 'MountainCarContinous-v0'.

As shown in the figure below, all agents exhibited oscillations in their reward-per-episode graphs, indicating that none consistently maintained a reward above 50 per episode for more than 10 consecutive episodes. This may be attributed to the differing dynamics of the environments. However, apart from Agent B, all models managed to achieve an average reward of over 50 per episode across 100 episodes, demonstrating good performance most of the time. Notably, Agent A, which used the original hyperparameters from the CartPole environment, outperformed the others. It had the shortest runtime

and achieved the highest average score across all attempts. We also experimented with different stopping criteria, but these did not lead to improved convergence.

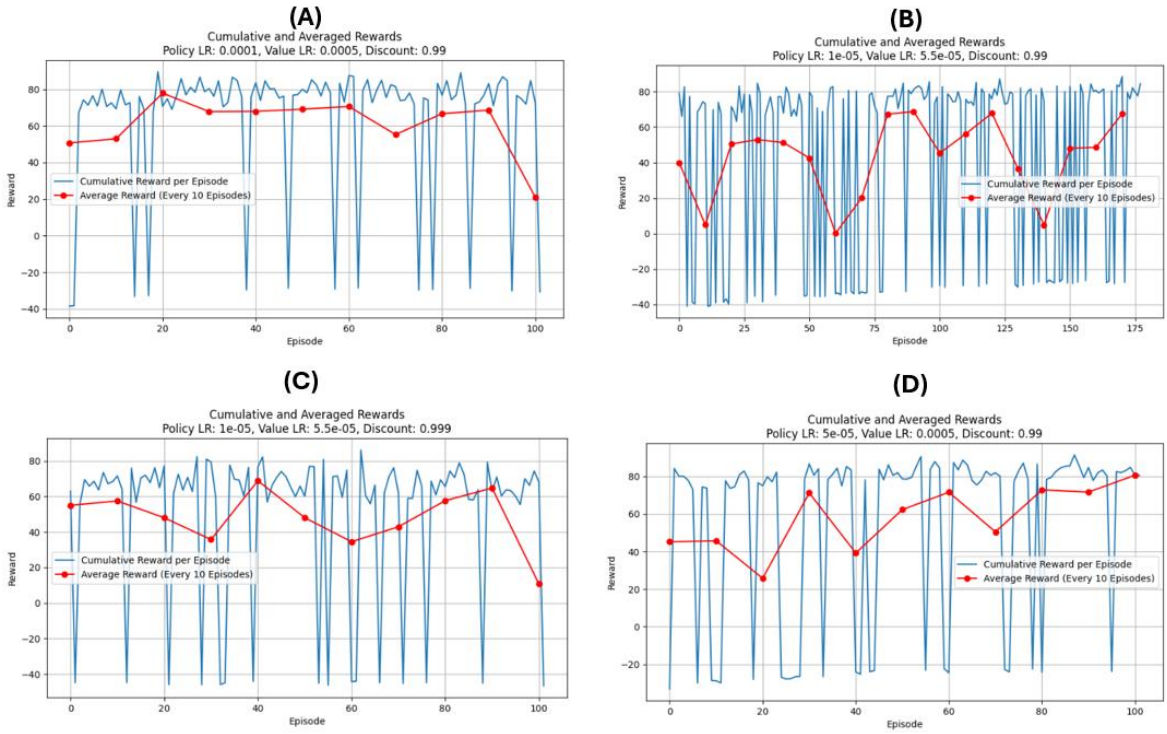


Figure 12: Hyper-parameterization results.

In summary, we tested a variety of configurations to develop the best-performing agent. The agent that yielded the best results utilized the hyperparameters from the CartPole environment. Despite this success, minor oscillations were still present, possibly due to the complexity of the environment. It is likely that a deeper network is required for this type of environment to capture its unique characteristics more effectively.

Section 3– transfer learning

In this section, we will train two progressive networks. The first network will leverage models previously trained on the Acrobot and MountainCarContinuous environments to optimize the agent for the CartPole environment. The second network will use models trained on the Acrobot and CartPole environments to optimize the agent for the MountainCarContinuous environment. For each agent, we will discuss the same key aspects as in the previous section: model architecture, hyperparameter tuning, and experimental results.

In this section, we implemented the progressive architecture only on the actor (policy) to leverage transferable skills like balancing and navigating across tasks. This approach allows the critic to adapt independently to the target environment's specific reward structure, avoiding instability and inefficiency. Focusing on the actor simplifies the architecture, accelerates learning, and ensures effective knowledge transfer.

1. Progressive network: Acrobot, MountCarcontinous -> 'Cartpole-v1':

Architecture:

Actor:

The models $h_i^{(1)}, h_i^{(2)}$ *have the same* architecture *as in section one. The* $h_i^{(3)}$ architecture is presented in table 13. The full model of the progressive net is represented in figure 10.

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**12**	**Relu**
2	**Linear**	**12**	**Relu**
3	**Linear**	**12**	**Softmax**

Table 13: The h(3) architecture..

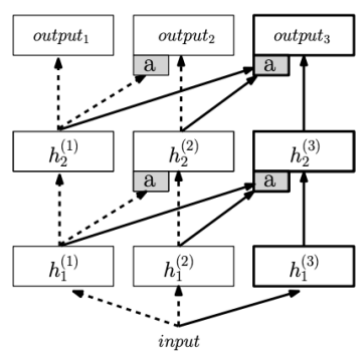


Figure 10: Actor's architecture.

Critic:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**64**	**Relu**
2	**Linear**	**64**	**Relu**
3	**Linear**	**64**	**-**

Table 14: Actor's architecture.

Hyperparameter tuning:

To optimize the agent's performance and achieve faster convergence, we experimented with several hyperparameter configurations, as detailed in **Table 15**. Initially, we applied the hyperparameters that were successful in training the agent on the 'CartPole-v1' environment. Each agent could learn until 1000 episodes and the max step per episode was defined as 500.

Hyper parameters	Original Cartpole-v1	Original Mountain-continous	Attempt 1	Attempt 2	Attempt 3
Mark	**A**	**B**	**C**	*****	**D**
Discount factor	**0.99**	**0.99**	**0.99**	**0.999**	**0.999**
Actor LR	**0.0001**	**0.0005**	**0.00005**	**0.00005**	**0.0008**
Critic LR	**0.0005**	**0.0005**	**0.0005**	**0.0005**	**0.0005**
Averaged Running time [sec]	**734**	**652**	**407**	**350**	**450**
Convergence (episodes)	**Didn't**	**Didn't**	**840**	**717**	**Didn't**

Table 15: Hyper-parameterization .

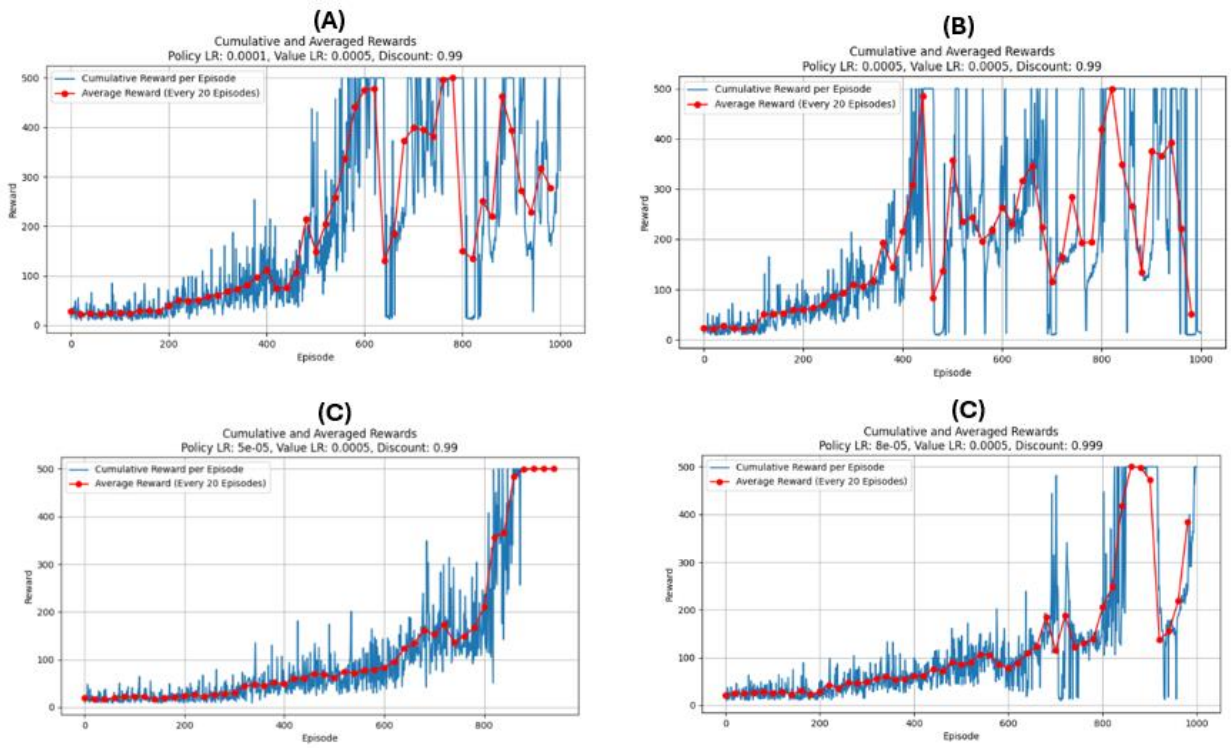


Figure 11: Hyper-parameterization results.

The agent with the poorest performance was Agent D, requiring 450 seconds of training for 1000 episodes. This indicates that the agent barely managed to achieve the maximum reward of 500 per episode. While some agents were able to reach the highest score of 500, their performance was inconsistent. Ultimately, the top-performing agent is highlighted in the results section.

Results:

Ultimately, we selected these hyperparameters because this agent outperformed the others, the third attempt. The chosen hyperparameters are:

- **Discount factor:** 0.999
- **Actor/Policy learning rate:** 0.00005
- **Critic/Value network learning rate:** 0.0005

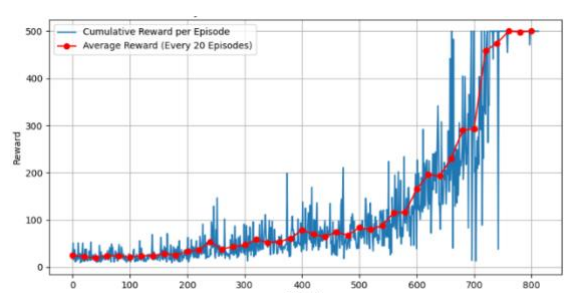


Figure 12: The best results

The agent achieved a reward of 500 for the first time after approximately 660 episodes. Compared to the basic model trained in the first section, this model trained and stabilized more quickly, reaching stability around 660 episodes, whereas the model in the first section required approximately 800 episodes to stabilize.

2. Progressive network: 'Acrobot-v1', 'CartPole-v1'-> 'MountCarContinous-v0':

Architecture:

Actor:

The models $h_i^{(1)}, h_i^{(2)}$ *have the same* architecture *as in section one. The* $h_i^{(3)}$ architecture is presented in table 13. The full model of the progressive net is represented in figure 10.

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**16**	**Relu**
2	**Linear**	**16**	**Relu**
3	**Linear**	**16**	**-**

Table 15: The h(3) architecture..

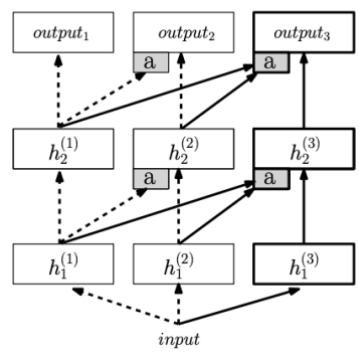


Figure 12: Actor's architecture.

Critic:

Layer	Type of layer	Layer's size	Activation function
1	**Linear**	**64**	**Relu**
2	**Linear**	**64**	**Relu**
3	**Linear**	**64**	**-**

Table 16: Critic's architecture.

Hyperparameter tuning:

To optimize the agent's performance and achieve faster convergence, we experimented with several hyperparameter configurations, as detailed in **Table 17**. Initially, we applied the hyperparameters that were successful in training the agent on the **'MountCarContinous-v0'** environment. Each agent could learn until 1000 episodes and the max step per episode was defined as 500.

Hyper parameters	Original Mountain-continous	Attempt 1	Attempt 2	Attempt 3	Attempt 4
Mark	**A**	*	**B**	**C**	**D**
Discount factor	**0.99**	**0.99**	**0.99**	**0.999**	**0.999**
Actor LR	**0.00001**	**0.0001**	**0.001**	**0.0001**	**0.001**
Critic LR	**0.00055**	**0.00055**	**0.0005**	**0.00055**	**0.0005**
Averaged Running time [sec]	**278**	**267**	**275**	**247**	**202**
Convergence (episodes)	**62**	**43**	**Didn't**	**Didn't**	**Didn't**

Table 15: Hyper-parameterization .

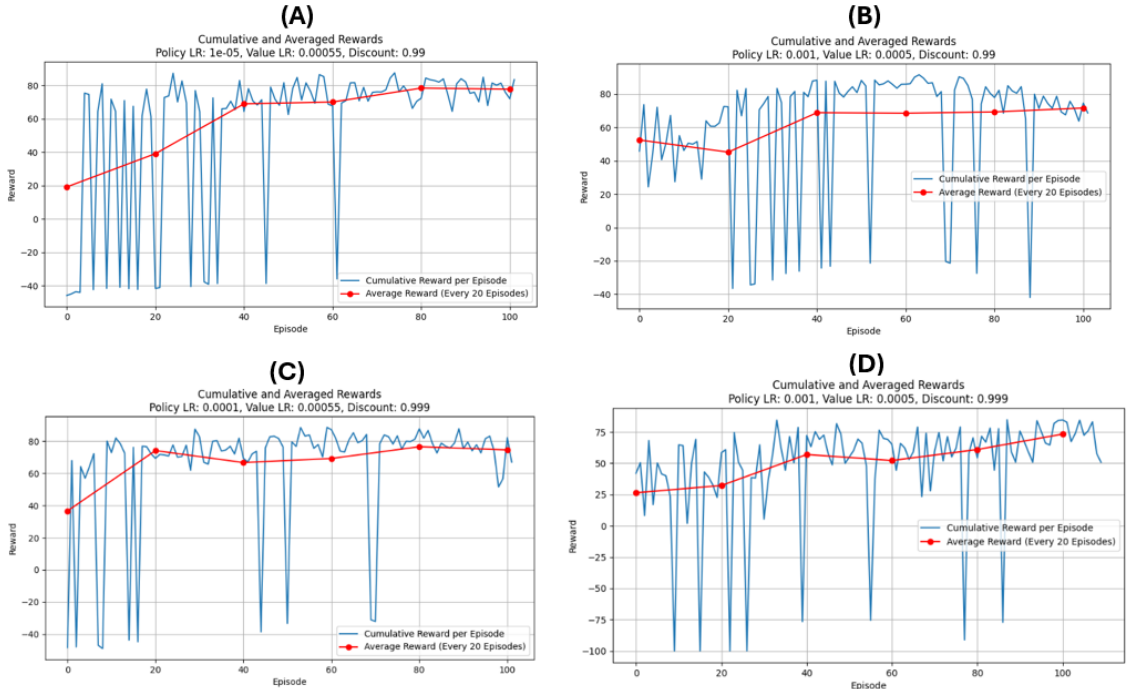


Figure 13: Hyper-parameterization results.

*As shown in the figure above, most hyper parameterization attempts failed to stabilize the system, resulting in fluctuations in the environment's score, except for configurations A and *.*

Results:

Ultimately, we selected these hyperparameters because this agent outperformed the others, the third attempt. The chosen hyperparameters are:

- **Discount factor:** 0.99
- **Actor/Policy learning rate:** 0.0001
- **Critic/Value network learning rate:** 0.00055

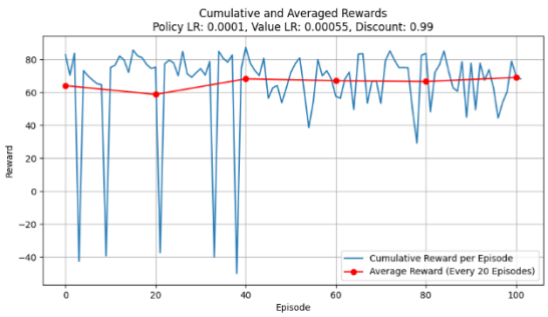


Figure 14: The best results

This score outperforms the others by achieving the shortest runtime, meaning the agent achieved the best score while remaining in the environment, leading to a lower overall score. Additionally, there are no oscillations, and the agent converged after approximately 40 episodes.

Running the scripts:

All the scripts for this assignment can be found either in the ZIP file that uploaded to the moodle, or in the following Git repo (branch: “Assignment#2”): https://github.com/oz182/DRL_Assignments.git